

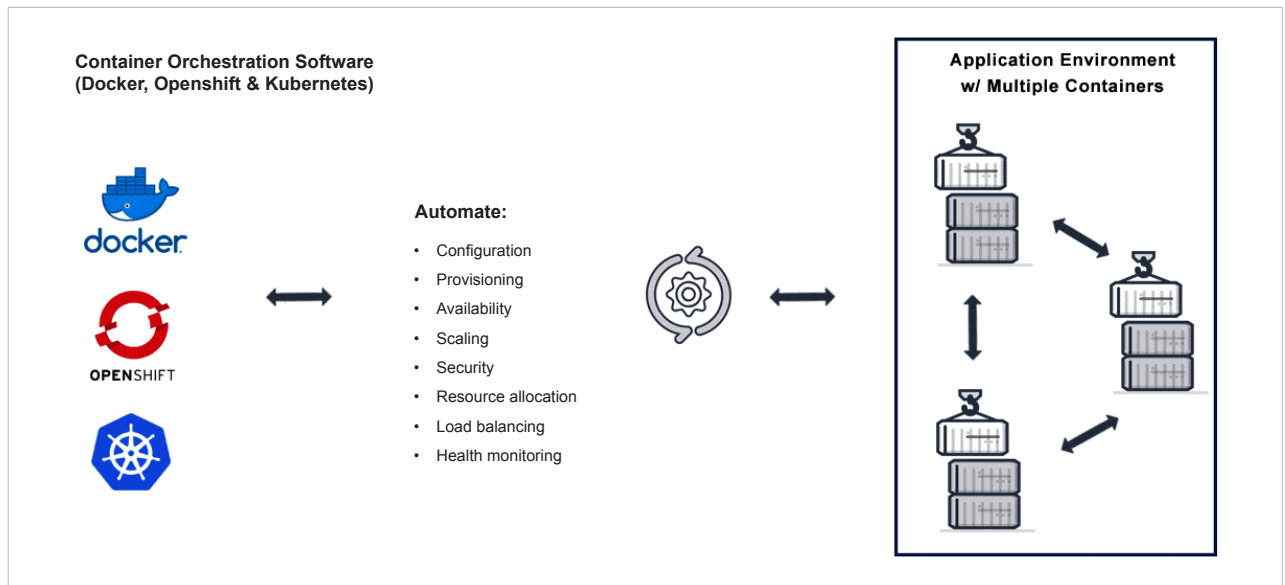


Figure 1: Container orchestration

When you use a container orchestration tool, such as Kubernetes or Docker Swarm, you typically describe the configuration of the application in a YAML or JSON file, depending on the tool. These configuration files (e.g., *docker-compose.yml*) tell the orchestration tool where to gather container images (e.g., Docker Hub), how to establish networking between the containers, how to mount storage volumes, and where to store logs for that container.

Containers are deployed on hosts, usually in replicated groups. When it's time to deploy a new container into a cluster, the container orchestration tool schedules the deployment and looks for the most appropriate host to place the container, based on predefined constraints. Once the container is running on the host, the orchestration tool manages its life cycle according to the specifications mentioned in the container's definition file.

There are lots of options for container orchestration like Docker Native 'Swarm', AWS Container Orchestrator, Elastic Container Service and Apache Mesos, a strong open source framework. Among all these

options, Apache Mesos has the unique capability to handle container and non-container workloads using a distributed channel in-built in the framework.

The advantages of container orchestration tools are:

- **Increased portability:** Application scalability with just a single command without impacting the entire application.
- **Simple and fast deployment:** Ease of creation of new containerised applications to handle increasing traffic load.
- **Enhanced productivity:** Simplified installation process and decreased dependency errors.
- **Improved security:** Application isolation improves Web application security by separating each application's process into different containers.

Figure 1 highlights the working of container orchestration.

How to select a container management and orchestration tool

Some of the variables which need to be considered for tool selection are listed below, but the final selection will

depend on the user's requirements.

- **CNI networking:** A good tool should allow trivial network connectivity between services to save developers time in writing special-purpose codes for finding dependencies.
- **Simplicity:** The tool should be easy to understand, and its interface should have all the options handy to perform real-time container operations.
- **Active development:** The tool should be under active development to fix the bugs and increase new options on regular intervals.
- **Cloud vendor:** The tool chosen should not be tied to any single cloud provider.

A deep dive into Kubernetes

Kubernetes has established itself as the *de facto* standard for container orchestration. It started as the flagship project of the Cloud Native Computing Foundation, which is backed by key players such as Google, Amazon Web Services (AWS), Microsoft, IBM, Intel, Cisco and Red Hat. Kubernetes continues to gain popularity with DevOps practitioners because it allows them to deliver a Platform-as-a-Service